DPS912 Lab 5: I/O Control

Due: Wednesday, June 10, 2026 (11:59pm)

In this lab, you will explore one of the most fundamental communication mechanisms in UNIX systems: **process signals**. Signals are how the operating system and applications notify each other about important events such as shutdowns, interrupts, or errors. They are lightweight, fast, and widely used in real-world system software.

You will build a **mini system monitor** that uses signals to coordinate multiple processes. A parent process will act as a controller, and two child processes will behave like network interface monitors. Each child represents a monitoring agent for a specific network interface, and the parent controls when monitoring begins and ends.

Rather than using shared memory, sockets, or pipes, all coordination in this lab happens through **signals only**. This will give you direct experience with:

- Synchronizing process start-up and shutdown
- Handling asynchronous events safely
- Controlling child behavior from a parent process
- Preventing unwanted termination from keyboard interrupts

What you will implement

In this lab you will use signals as a method of communication between a parent process and its children processes. Signals will be used to synchronize the start-up and shutdown of all children processes.

The particular application in mind is a network monitor. The parent will start up children to monitor the network interfaces on a machine. In our case, there are two interfaces: **ens33** and **lo**. There will therefore be two children. One child will monitor **ens33** and one child will monitor **lo**.

The parent process (**sysmonExec**) will spawn two children (**intfMonitor**) as follows:
$ ./intfMonitor lo
$ ./intfMonitor ens33

For the parent, you will have to send the start signal (SIGUSR1) to the children, sleep for 30 seconds, then send the stop signal (SIGUSR2) to the children. The parent should wait for all children to shutdown before shutting itself down.

For the child, you will have to register signal handlers for start-up (SIGUSR1), shutdown (SIGUSR2), ctrl-C and ctrl-Z. For ctrl-C and ctrl-Z, your signal handler will simply discard them, meaning your program will not shutdown on ctrl-C nor be put in the background on ctrl-Z. Your child will have to

wait until it receives a start signal from the parent before starting. The child's signal handler will handle 4 signals as follows:

- If the signal handler receives a SIGUSR1, the following message should appear on the screen:
  intfMonitor: starting up
- If the signal handler receives a ctrl-C, the following message should appear on the screen:
  intfMonitor: ctrl-C discarded
- If the signal handler receives a ctrl-Z, the following message should appear on the screen:
  intfMonitor: ctrl-Z discarded
- If the signal handler receives a SIGUSR2, the following message should appear on the screen:
  intfMonitor: shutting down
- If the signal handler receives any other signal, the following message should appear on the screen:
  intfMonitor: undefined signal

When the child receives the shutdown signal, it should stop processing and exit.

Code has been given to you so you can concentrate on the signals portion only. Simply fill in the parts indicated by **TODO**.

- The Makefile can be found at Makefile.
  The code for the parent can be found at sysmonExec.cpp.
  The code for the child can be found at intfMonitor.cpp.

Answer the following Research Questions:

1) Research what a zombie process is. Why does the parent process in this lab need to call wait() or waitpid() after sending the shutdown signal?
2) In this lab, each child process monitors one network interface. Research one Linux command or file that can be used to inspect network interface statistics, such as /sys/class/net, ip -s link, or ifconfig. What information can be collected from it?

Assignment Submission:

- Complete all steps, Add all output-screenshot and explanations (if required) to a MS-Word file.
- Add the following declaration at the top of MSWORD file
  /**
  ***
  * DPS912-Lab5
  * I declare that this lab is my own work in accordance with Seneca Academic Policy.
  * No part of this assignment has been copied manually or electronically from any other source
  * (including web sites) or distributed to other students.
  *

```
* Name: _____ Student ID: _____ Date: _____
*
*
*************************************************************************
**/
```

- Please submit the Source code (zip all .c, .h, and makeFiles)
- Please answer the following two declarations:
    - D1) On a scale from 1 to 5, **How much did you use generative AI to complete this assignment?**
        - where:
        - **1** means you did not use generative AI at all
        - **2** means you used it very minimally
        - **3** means you used it moderately
        - **4** means you used it significantly
        - **5** means you relied on it almost entirely
        - **Your answer :**
    - D2) On a scale from 1 to 5, **How confident are you in your understanding of the generative AI support you utilized in this assignment, and in your ability to explain it if questioned?**
        - where:
        - **1** means "Not confident at all – I do not understand the generative AI support I used and cannot explain it."
        - **2** means "Slightly confident – I understand a little, but I have many uncertainties."
        - **3** means "Moderately confident – I understand the majority of the support, though some parts are unclear."
        - **4** means "Very confident – I understand most of the AI support well and can explain it with minor gaps."
        - **5** means "Extremely confident – I fully understand the generative AI support I used and can clearly explain or justify it if asked."
        - **Your answer :**

Important Note:

- **LATE SUBMISSIONS for labs**. Late lab submissions will receive a **10% deduction per day**. Labs submitted more than **three days late will receive a grade of zero (0)**.
    - If you are unable to finish the lab on time, **submit whatever you have before the deadline** to avoid an automatic zero.
- This labs should be submitted along with a video-recording which contains a detailed walkthrough of solution. Without recording, the assignment can get a maximum of 1/3 of the total.

  - Note: In case you are running out of time to record the video, you can submit the assignment (source code + screenshots) by the deadline and submit the video within 24 hours after the deadline.

-

Important Note:

- **LATE SUBMISSIONS for labs**. There is a deduction of 10% for Late assignment submissions, and after three days it will grade of zero (0).
- This labs should be submitted along with a video-recording which contains a detailed walkthrough of solution. Without recording, the assignment can get a maximum of 1/3 of the total.
  - Note: In case you are running out of time to record the video, you can submit the assignment (source code + screenshots) by the deadline and submit the video within 24 hours after the deadline.